

■ Conditional Statements

Let your program **decide** what to do based on **True / False** conditions.

Keyword	What it does
if	Runs a block only if the condition is True
elif	('else if') checks another condition if previous was False
else	Runs if ALL above conditions were False

Example 1 — Age Check

```
age = int(input('enter age: '))
if age >= 18:
    print('you can vote')
    print('you can drive')
else:
    print("you can't vote")
    print("you can't drive")
```

Example 2 — Traffic Lights (elif chain)

```
color = input('enter color: ')
if color == 'red':
    print('Stop')
elif color == 'yellow':
    print('Look')
elif color == 'green':
    print('Go')
else:
    print('wrong color')
```

■ *elif & else can NOT be used without a preceding if statement.*

Logical Operators in Conditions

Combine multiple conditions with **and**, **or**, **not**.

```
if age < 13:
    print('child')
elif age >= 13 and age < 18:
    print('teenager')
else:
    print('adult')
```

Nested Conditionals

Put one **if** inside another to make decisions inside decisions.

🌸 Python Fundamentals 🌸 Part 2

Conditional Statements • Loops • Functions

```
# Login System
username = input('enter username: ')
password = input('enter password: ')
if username == 'admin' and password == 'pass':
    print('log in successful!')
else:
    if username != 'admin': # NESTING
        print('wrong username, try again.')
    else:
        print('wrong password, try again.')
```

Ternary Statements — one-liner if-else

A compact way to write simple if-else in a single line:

```
# Syntax
value_if_true if condition else value_if_false

# Example
age = 18
status = 'Adult' if age >= 18 else 'Not Adult'
print(status) # Adult
```

■ *Use ternary only for simple conditions — avoid for nested/complex ones!*

Match-Case Statements (Python 3.10+)

A cleaner alternative to long if-elif-else chains. Rarely used in production.

```
match variable:
    case pattern1:
        # code for pattern1
    case pattern2:
        # code for pattern2
    case _: # wildcard = 'default'
        # default code

# Traffic light example
match color:
    case 'red': print('Stop')
    case 'yellow': print('Look')
    case 'green': print('Go')
    case _: print('wrong color')
```

■ Practice Problems — Set 1

#	Problem
1	For a given number n, print if it's a multiple of 5 or not
2	For a given number n, print if it's Odd or Even

■ Loops

A **loop** lets us execute a block of code multiple times.

Loop type	When to use
while	Repeat while a condition is True
for	Iterate over a sequence (list, string, range ...)

while Loop

```
while condition:
    # code block

# Print 1 → 5
i = 1
while i <= 5:
    print(i)
    i += 1

# Print 5 → 1
i = 5
while i > 0:
    print(i)
    i -= 1
```

■ If the condition is always True → infinite loop! Always make sure *i* updates inside the loop.

Loop Control Statements

Keyword	What it does
break	Stops the loop immediately
continue	Skips rest of current iteration, moves to next

```
# break – stops at first multiple of 6
i = 1
while i <= 10:
    if i % 6 == 0:
        break
    print(i) # output: 1 2 3 4 5
    i += 1

# continue – skips multiples of 3
i = 0
while i < 10:
    i += 1
    if i % 3 == 0:
        continue
    print(i) # output: 1 2 4 5 7 8 10
```

for Loop

```
for variable in sequence:
    # code for each item

# Basic range
for i in range(5):
    print(i) # 0 1 2 3 4

# Looping over a string
word = 'Prime'
for ch in word:
    print(ch) # P r i m e

# Count i's in a word
word = 'artificial intelligence'
count = 0
for ch in word:
    if ch == 'i':
        count += 1
print(f'i occurs {count} times.')
```

Nested for Loops

```
for i in range(1, 3):
    for j in range(1, 3):
        print(f'({i}, {j})') # (1,1)(1,2)(2,1)(2,2)
```

range() Function

Generates a sequence of numbers — super memory-efficient (uses an iterator).

Parameter	Default	Meaning
start	0	Number to start from
stop	—	Stop BEFORE this number (not included)
step	1	How much to increase each time

```
range(5) # 0,1,2,3,4
range(1, 6) # 1,2,3,4,5
range(1, 10, 2) # 1,3,5,7,9
```

■ Practice Problems — Set 2

#	Problem	Use
1	Multiplication table for n	while
2	Print odd numbers 1–10 (use continue)	while
3	Count vowels in a word	for

4	Sum of first n natural numbers	for
---	--------------------------------	-----

■ Functions

Functions are reusable blocks of code that perform a specific task — they keep your code **organised & readable**.

Defining & Calling

```
# Define with 'def'
def hello():
    print('hello!')

hello() # Call the function
```

Parameters vs Arguments

Parameters — names in the function definition. **Arguments** — actual values passed when calling.

```
def sum(a, b): # a, b are PARAMETERS
    print(a + b)

sum(5, 10) # 5, 10 are ARGUMENTS
```

Return Values

```
def avg(a, b, c):
    return (a + b + c) / 3

print(avg(1, 2, 3)) # 2.0
```

Default Parameters

Assign a fallback value — used when the caller doesn't provide that argument. **Default params must come last!**

```
def sum(a, b=1): # b defaults to 1
    return a + b

print(sum(5)) # 6 (b used default)
print(sum(5, 3)) # 8 (b overridden)
```

Lambda Functions (anonymous one-liners)

Short functions created with the **lambda** keyword — no name, single expression.

🌸 Python Fundamentals 🌸 Part 2

Conditional Statements • Loops • Functions

```
# Syntax
lambda arguments: expression

# Square a number
square = lambda x: x * x
print(square(5)) # 25
```

Built-in Functions (already know these!)

Function	What it does
print()	Outputs to console
input()	Reads user input (returns string)
type()	Returns the data type of a value
range()	Generates a sequence of numbers
int()	Converts a value to integer
len()	Returns length of a sequence

■ Practice Problems — Set 3

#	Problem
1	Create a function to compute factorial of n
2	Create a function to return largest of 3 numbers a, b, c

■ Solutions

Set 1 — Conditionals

1. Multiple of 5?

```
num = int(input('enter number: '))
if num % 5 == 0:
    print('multiple of 5')
else:
    print('NOT a multiple of 5')
```

2. Odd or Even?

```
num = int(input('enter number: '))
if num % 2 == 0:
    print('Even')
else:
    print('Odd')
```

Set 2 — Loops

1. Multiplication table of n

🌸 Python Fundamentals 🌸 Part 2

Conditional Statements • Loops • Functions

```
n = int(input('enter n: '))
i = 1
while i <= 10:
    print(i * n)
    i += 1
```

2. Odd numbers 1–10 (continue)

```
i = 0
while i < 10:
    i += 1
    if i % 2 == 0:
        continue
    print(i) # 1 3 5 7 9
```

3. Count vowels

```
count = 0
for ch in word:
    if ch in 'aeiouAEIOU':
        count += 1
print(f'vowel count = {count}')
```

4. Sum of first n natural numbers

```
n = int(input('enter n: '))
total = 0
for i in range(1, n + 1):
    total += i
print('sum =', total)
```

Set 3 — Functions

1. Factorial of n

```
def factorial(n):
    result = 1
    for i in range(1, n + 1):
        result *= i
    return result

print(factorial(5)) # 120
```

2. Largest of 3 numbers

🌸 Python Fundamentals 🌸 Part 2

Conditional Statements • Loops • Functions

```
def get_largest(a, b, c):  
    if a > b and a > c:  
        return a  
    elif b > c:  
        return b  
    else:  
        return c  
  
print(get_largest(3, 10, 5)) # 10
```

■ Quick Cheat Sheet

Concept	Syntax / Key points
if / elif / else	if cond: ... elif cond: ... else: ...
Ternary	val_true if condition else val_false
match-case	match var: case p1: ... case _: (default)
while loop	while condition: ... (don't forget to update it!)
for loop	for var in sequence: ...
range()	range(start=0, stop, step=1)
break	Exits the loop immediately
continue	Skips current iteration, goes to next
def function	def name(params): ... return value
default param	def fn(a, b=10) — defaults must be last
lambda	lambda x: expression

■ *You've got this! Code is just logic — and you're already logical* ■